Web Development Notes

A complete beginner-to-intermediate reference

HTML | CSS | JavaScript | Git | Backend | APIs | Databases | Security | Deployment

Contents

1. How the Web Works

2. HTML

3. CSS

4. JavaScript

5. Responsive Design and Accessibility

6. Git and GitHub

7. Backend Development (Node.js and Express)

8. APIs and REST

9. Databases and SQL

10. Authentication and Web Security

11. Frameworks, Tools and Workflow

12. Deployment, Performance and SEO

13. Learning Roadmap and Cheat Sheets

1. How the Web Works

1.1 What is web development?

Web development is the process of building websites and web applications that run in a browser. It is usually split into three areas:

- **Front-end** - what the user sees and interacts with (HTML, CSS, JavaScript).
- **Back-end** - the server, application logic, databases and APIs behind the scenes.
- **Full-stack** - working on both front-end and back-end.

1.2 Client-server model

A **client** (usually a browser) sends a **request** to a **server**. The server processes it and sends back a **response** (HTML, JSON, images, etc.).

1.3 What happens when you open a website

- You type a URL such as https://example.com/about in the browser.
- The browser asks **DNS** (the internet's phonebook) for the server's IP address.
- The browser opens a connection to that IP (using TCP, and TLS for HTTPS).
- It sends an **HTTP request**: GET /about.
- The server replies with an **HTTP response** containing HTML.
- The browser parses the HTML, requests linked CSS, JS and images, then renders the page.

1.4 Anatomy of a URL

```
https://www.example.com:443/blog/post?id=7&lang=en#comments
|_____|  |_____________||___||________||___________||_______|
scheme       host       port   path        query      fragment
```

1.5 HTTP basics

Method	Purpose	Example
GET	Read data	GET /users
POST	Create data	POST /users
PUT / PATCH	Replace / partially update data	PATCH /users/5
DELETE	Remove data	DELETE /users/5

Status code	Meaning
200 OK	Request succeeded
201 Created	New resource created
301 / 302	Redirect (permanent / temporary)
400 Bad Request	Client sent invalid data
401 Unauthorized	Not logged in / invalid credentials

Status code	Meaning
403 Forbidden	Logged in but not allowed
404 Not Found	Resource does not exist
500 Internal Server Error	Server-side bug or failure

1.6 Key terms

- **Domain** - human-readable address (example.com).
- **Hosting** - a server where your site's files live.
- **HTTPS** - HTTP encrypted with TLS; always use it.
- **Static site** - fixed files served as-is. **Dynamic site** - pages generated per request.
- **Browser DevTools** - press F12 to inspect HTML, CSS, network requests and console.

2. HTML (HyperText Markup Language)

HTML describes the **structure and meaning** of a web page using elements written as tags.

2.1 Basic page skeleton

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello, World!</h1>
  <script src="app.js"></script>
</body>
</html>
```

2.2 Common elements

Element	Use
h1 - h6	Headings (h1 is most important; use one per page)
p	Paragraph
a href="..."	Link
img src alt	Image (always give alt text)
ul / ol / li	Unordered / ordered lists
div / span	Generic block / inline containers
table, tr, th, td	Tabular data
form, input, button	User input
strong / em	Important text / emphasised text

```
<a href="https://example.com" target="_blank" rel="noopener">Visit</a>
<img src="cat.jpg" alt="A grey cat sleeping on a sofa" width="300">
<ul>
  <li>HTML</li>
  <li>CSS</li>
</ul>
```

2.3 Semantic HTML

Semantic tags describe their purpose, which helps accessibility, SEO and maintainability.

```
<header>   site title, logo, navigation
<nav>      main navigation links
<main>     the main content (one per page)
<section>  a thematic group of content
<article>  self-contained content (blog post, news item)
<aside>    side content (sidebar, related links)
<footer>   copyright, contacts, links
```

2.4 Forms

```
<form action="/signup" method="POST">
  <label for="email">Email</label>
  <input type="email" id="email" name="email" required>

  <label for="pw">Password</label>
  <input type="password" id="pw" name="password" minlength="8" required>

  <select name="country">
    <option value="ke">Kenya</option>
    <option value="ug">Uganda</option>
  </select>

  <textarea name="bio" rows="3"></textarea>
  <input type="checkbox" name="terms"> I agree
  <button type="submit">Sign up</button>
</form>
```

Common input types: text, email, password, number, date, checkbox, radio, file, range, color, search, tel, url. Always pair inputs with a **label**.

2.5 Useful attributes

- **id** (unique) and **class** (reusable) - hooks for CSS and JavaScript.
- **data-*** - custom data, e.g. data-user-id="5".
- **placeholder, required, disabled, readonly, autofocus** - form behaviour.
- **title, lang, hidden, tabindex** - general attributes.

3. CSS (Cascading Style Sheets)

CSS controls the **presentation**: colours, fonts, spacing and layout.

3.1 Syntax and ways to add CSS

```
selector {
  property: value;
}

p { color: #333; font-size: 16px; }
```

- **External** (best): link rel="stylesheet" href="style.css".
- **Internal**: a style element in the head.
- **Inline**: style="color:red" on an element (avoid except for quick tests).

3.2 Selectors

Selector	Example	Selects
Element	p	All paragraphs
Class	.card	Elements with class="card"
ID	#header	The element with id="header"
Descendant	nav a	Links inside nav
Child	ul > li	Direct children only
Group	h1, h2	Both h1 and h2
Attribute	input[type="text"]	Text inputs
Pseudo-class	a:hover, li:first-child	State or position
Pseudo-element	p::first-line, div::after	Part of an element

3.3 Specificity and the cascade

When rules conflict, the browser picks by: **importance (!important)**, then **specificity** (inline > ID > class/attribute/pseudo-class > element), then **source order** (later wins). Avoid !important and overly specific selectors.

3.4 The box model

Every element is a box: **content**, surrounded by **padding**, then **border**, then **margin**.

```
* { box-sizing: border-box; }   /* width includes padding and border */

.box {
  width: 300px;
  padding: 16px;
  border: 2px solid #333;
  margin: 20px auto;           /* auto centres a block horizontally */
}
```

3.5 Colours, units and typography

```
color: red;  color: #1f4e79;  color: rgb(31 78 121);  color: hsl(210 59% 30%);

/* Units */
px   fixed pixels
%    relative to parent
em   relative to parent font size
rem  relative to root font size (recommended for text)
vw, vh   1% of viewport width / height

body {
  font-family: "Inter", Arial, sans-serif;
  font-size: 1rem;
  line-height: 1.6;
  text-align: left;
}
```

3.6 Display and position

```
display: block | inline | inline-block | none | flex | grid;

position: static;    /* default */
position: relative;  /* offset from its normal place */
position: absolute;  /* relative to nearest positioned ancestor */
position: fixed;     /* stays on screen when scrolling */
position: sticky;    /* sticks after scrolling to a threshold */
z-index: 10;         /* stacking order for positioned elements */
```

3.7 Flexbox (one-dimensional layout)

```
.container {
  display: flex;
  flex-direction: row;            /* or column */
  justify-content: space-between; /* main axis */
  align-items: center;            /* cross axis */
  flex-wrap: wrap;
  gap: 1rem;
}
.item { flex: 1; }                /* grow equally */
```

3.8 CSS Grid (two-dimensional layout)

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
.wide { grid-column: span 2; }

/* Responsive grid with no media queries */
.cards { display: grid;
         grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
         gap: 1rem; }
```

3.9 Transitions, animations and variables

```
:root { --brand: #1f4e79; --radius: 8px; }

.btn {
  background: var(--brand);
  border-radius: var(--radius);
  transition: transform 0.2s ease, background 0.2s;
}
.btn:hover { transform: translateY(-2px); }
```

```
@keyframes fade { from { opacity: 0; } to { opacity: 1; } }
.modal { animation: fade 0.4s ease-in; }
```

4. JavaScript

JavaScript makes pages **interactive**. It runs in the browser (and on servers via Node.js).

4.1 Variables and data types

```
let age = 25;            // can be reassigned
const name = "Amina";    // cannot be reassigned (preferred default)
// avoid var

// Types: string, number, boolean, null, undefined, bigint, symbol, object
typeof "hi";             // "string"
const list = [1, 2, 3];  // array
const user = { name: "Amina", age: 25 };   // object
```

4.2 Operators and control flow

```
// === strict equality (use this), !== strict inequality
if (age >= 18 && name) {
  console.log("Adult");
} else if (age > 12) {
  console.log("Teen");
} else {
  console.log("Child");
}

const label = age >= 18 ? "Adult" : "Minor";   // ternary
const city = user.city ?? "Unknown";           // nullish coalescing
const zip = user.address?.zip;                 // optional chaining

for (let i = 0; i < 3; i++) console.log(i);
for (const item of list) console.log(item);
while (age < 30) { age++; }
```

4.3 Functions

```
function add(a, b) { return a + b; }

const multiply = (a, b) => a * b;          // arrow function

function greet(name = "friend") {          // default parameter
  return `Hello, ${name}!`;                // template literal
}
```

4.4 Arrays and objects

```
const nums = [1, 2, 3, 4, 5];
nums.push(6);  nums.pop();

nums.map(n => n * 2);              // [2, 4, 6, 8, 10]
nums.filter(n => n % 2 === 0);     // [2, 4]
nums.reduce((sum, n) => sum + n, 0);  // 15
nums.find(n => n > 3);             // 4
nums.includes(3);                  // true

// Destructuring and spread
const [first, ...rest] = nums;
const { name, age } = user;
const copy = { ...user, age: 26 };
const merged = [...nums, 99];
```

4.5 The DOM (Document Object Model)

```
const title = document.querySelector("h1");
const items = document.querySelectorAll(".item");

title.textContent = "New title";
title.classList.add("active");
title.style.color = "red";

const li = document.createElement("li");
li.textContent = "New item";
document.querySelector("ul").appendChild(li);
```

4.6 Events

```
const btn = document.querySelector("#btn");

btn.addEventListener("click", (event) => {
  console.log("Clicked!", event.target);
});

form.addEventListener("submit", (e) => {
  e.preventDefault();                  // stop page reload
  const email = form.elements.email.value;
});
```

Common events: click, input, change, submit, keydown, keyup, mouseover, scroll, DOMContentLoaded, load. Use **event delegation** (one listener on a parent) for long lists.

4.7 Asynchronous JavaScript

Some tasks (network requests, timers) take time. JavaScript handles them without blocking the page.

```
setTimeout(() => console.log("after 1s"), 1000);

// Promises with fetch
fetch("https://api.example.com/users")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));

// async / await (cleaner)
async function loadUsers() {
  try {
    const res = await fetch("/api/users");
    if (!res.ok) throw new Error("HTTP " + res.status);
    const users = await res.json();
    return users;
  } catch (err) {
    console.error("Failed:", err);
  }
}

// POST JSON
await fetch("/api/users", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Amina" })
});
```

4.8 Modules, storage and JSON

```
// math.js
```

```
export const add = (a, b) => a + b;
export default function square(x) { return x * x; }

// main.js
import square, { add } from "./math.js";     // use <script type="module">

localStorage.setItem("theme", "dark");        // persists
localStorage.getItem("theme");
sessionStorage.setItem("step", "2");          // cleared on tab close

const text = JSON.stringify({ a: 1 });        // object -> string
const obj = JSON.parse(text);                 // string -> object
```

4.9 Classes and error handling

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { return `${this.name} makes a sound`; }
}
class Dog extends Animal {
  speak() { return `${this.name} barks`; }
}

try { riskyCall(); }
catch (err) { console.error(err.message); }
finally { console.log("always runs"); }
```

Tip: Open the browser console (F12) and experiment. console.log() is your best debugging friend.

5. Responsive Design and Accessibility

5.1 Responsive design

Your site should work on phones, tablets and desktops. Use a **mobile-first** approach: write base styles for small screens, then add media queries for larger ones.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">

.container { padding: 1rem; }

@media (min-width: 768px) {          /* tablets and up */
  .container { max-width: 720px; margin: auto; }
}
@media (min-width: 1024px) {         /* desktops */
  .container { max-width: 1000px; }
}

img { max-width: 100%; height: auto; }
```

- Prefer flexible units (%, rem, fr, vw) over fixed pixel widths.
- Use flexbox and grid for layouts that adapt automatically.
- Use srcset and the picture element for responsive images.
- Test with DevTools device mode and on real devices.

5.2 Accessibility (a11y)

- Use **semantic HTML** first; add ARIA only when needed.
- Give every image meaningful **alt** text (empty alt for decorative images).
- Ensure sufficient **colour contrast** (WCAG AA: 4.5:1 for normal text).
- Make everything usable by **keyboard** (Tab, Enter, Space) with visible focus styles.
- Associate form **labels** with inputs; show clear error messages.
- Do not rely on colour alone to convey meaning.
- Use proper heading order (h1, then h2, then h3).

6. Git and GitHub

Git tracks changes to your code so you can undo mistakes and collaborate. **GitHub** hosts Git repositories online.

6.1 Everyday commands

```
git init                          # start a repo
git clone <url>                   # copy a remote repo
git status                        # what changed?
git add .                         # stage all changes
git commit -m "Add navbar"        # save a snapshot
git log --oneline                 # history
git diff                          # see unstaged changes

git branch feature-login          # create branch
git switch feature-login          # move to it
git merge feature-login           # merge into current branch

git remote add origin <url>
git push -u origin main           # upload
git pull                          # download and merge
```

6.2 Typical workflow

- Create a branch for each feature or fix.
- Commit small, focused changes with clear messages.
- Push the branch and open a **Pull Request** on GitHub.
- Get a review, then merge into main.
- Resolve **merge conflicts** by editing the marked files, then add and commit.

6.3 .gitignore

```
node_modules/
.env
dist/
.DS_Store
```

> **Tip:** Never commit secrets (API keys, passwords, .env files).

7. Backend Development (Node.js and Express)

The backend handles business logic, talks to the database, and exposes data to the front-end. Popular choices: Node.js (JavaScript), Python (Django, Flask, FastAPI), PHP (Laravel), Java (Spring), C# (.NET), Ruby (Rails), Go.

7.1 Node.js basics

```
node -v                  # check installation
npm init -y              # create package.json
npm install express      # install a package
npm install -D nodemon   # dev dependency
npm run dev              # run a script from package.json
```

7.2 A simple Express server

```
const express = require("express");
const app = express();

app.use(express.json());              // parse JSON bodies
app.use(express.static("public"));    // serve static files

app.get("/", (req, res) => res.send("Hello from the server"));

app.get("/api/users/:id", (req, res) => {
  res.json({ id: req.params.id, query: req.query });
});

app.post("/api/users", (req, res) => {
  const { name } = req.body;
  if (!name) return res.status(400).json({ error: "Name required" });
  res.status(201).json({ id: 1, name });
});

// Error-handling middleware (last)
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({ error: "Server error" });
});

app.listen(3000, () => console.log("Running on http://localhost:3000"));
```

7.3 Middleware

Middleware are functions that run between the request and the response. They can log, authenticate, validate, parse data or handle errors. Call next() to pass control on.

```
function logger(req, res, next) {
  console.log(req.method, req.url);
  next();
}
app.use(logger);
```

7.4 Environment variables

```
# .env  (never commit this)
PORT=3000
DB_PASSWORD=supersecret

// in code (using the dotenv package)
require("dotenv").config();
```

```
const port = process.env.PORT || 3000;
```

7.5 Recommended project structure

```
project/
  src/
    routes/        URL definitions
    controllers/   request handling logic
    models/        database access
    middleware/    auth, validation, logging
    config/
  public/          static assets
  .env
  package.json
```

8. APIs and REST

An **API** (Application Programming Interface) lets programs talk to each other. Web APIs usually exchange **JSON** over HTTP.

8.1 REST conventions

Action	Method and URL	Success code
List all	GET /api/posts	200
Get one	GET /api/posts/12	200
Create	POST /api/posts	201
Update	PUT or PATCH /api/posts/12	200
Delete	DELETE /api/posts/12	204

- Use **nouns** (plural) in URLs, not verbs: /users, not /getUsers.
- Use the right HTTP method and status codes.
- Version your API: /api/v1/users.
- Support pagination: /posts?page=2&limit;=20, plus filtering and sorting.
- Return consistent error shapes, e.g. { "error": "Message" }.

8.2 JSON example

```
{
  "id": 12,
  "title": "Learning APIs",
  "published": true,
  "tags": ["web", "api"],
  "author": { "id": 3, "name": "Amina" }
}
```

8.3 CORS

Browsers block requests from one origin (domain, protocol, port) to another unless the server allows it with CORS headers. In Express: npm install cors, then app.use(cors()) - restrict allowed origins in production.

8.4 Other API styles

- **GraphQL** - client asks for exactly the fields it needs from a single endpoint.
- **WebSockets** - persistent two-way connection (chat, live updates).
- **Webhooks** - a server calls your URL when an event happens.

8.5 Testing APIs

Use tools like Postman, Insomnia, Thunder Client (VS Code) or curl:

```
curl -X POST http://localhost:3000/api/users \
  -H "Content-Type: application/json" \
  -d '{"name":"Amina"}'
```

9. Databases and SQL

A database stores data permanently and lets you query it efficiently.

Type	Examples	Best for
Relational (SQL)	MySQL, PostgreSQL, SQLite	Structured data with relationships
Document (NoSQL)	MongoDB, Firestore	Flexible, nested JSON-like data
Key-value	Redis	Caching, sessions, fast lookups

9.1 Core SQL

```
CREATE DATABASE shop;
USE shop;

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO users (name, email) VALUES ('Amina', 'amina@example.com');

SELECT id, name FROM users WHERE name LIKE 'A%' ORDER BY name LIMIT 10;

UPDATE users SET name = 'Amina K.' WHERE id = 1;

DELETE FROM users WHERE id = 1;
```

9.2 Relationships and JOINs

```
CREATE TABLE orders (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  total DECIMAL(10,2),
  FOREIGN KEY (user_id) REFERENCES users(id)
);

SELECT u.name, o.id, o.total
FROM users u
INNER JOIN orders o ON o.user_id = u.id;     -- only matching rows

-- LEFT JOIN keeps all users, even those with no orders
-- Relationships: one-to-one, one-to-many, many-to-many (junction table)
```

9.3 Aggregation

```
SELECT user_id, COUNT(*) AS orders, SUM(total) AS spent
FROM orders
GROUP BY user_id
HAVING SUM(total) > 100
ORDER BY spent DESC;
```

9.4 Good practice

- Every table gets a **primary key**; link tables with **foreign keys**.
- **Normalise** data: avoid repeating the same information in many places.

- Add **indexes** to columns used often in WHERE, JOIN and ORDER BY.
- Always use **parameterised queries** (prepared statements) to prevent SQL injection.
- Use **transactions** (BEGIN, COMMIT, ROLLBACK) when several changes must succeed together.
- Back up your database regularly.

9.5 Using MySQL from Node.js

```
const mysql = require("mysql2/promise");
const pool = mysql.createPool({
  host: "localhost", user: "root",
  password: process.env.DB_PASSWORD, database: "shop"
});

app.get("/api/users/:id", async (req, res) => {
  const [rows] = await pool.execute(
    "SELECT id, name FROM users WHERE id = ?",   // ? placeholder: safe
    [req.params.id]
  );
  if (!rows.length) return res.status(404).json({ error: "Not found" });
  res.json(rows[0]);
});
```

10. Authentication and Web Security

10.1 Authentication vs authorization

- **Authentication** - who are you? (login)
- **Authorization** - what are you allowed to do? (roles, permissions)

10.2 Password handling

- Never store plain-text passwords. Hash with **bcrypt** or **argon2** (salted, slow by design).
- Enforce sensible password rules and consider multi-factor authentication.

```
const bcrypt = require("bcrypt");
const hash = await bcrypt.hash(password, 12);        // on signup
const ok = await bcrypt.compare(password, hash);     // on login
```

10.3 Sessions vs tokens

Approach	How it works	Notes
Session cookie	Server stores session; browser holds session ID cookie	Simple and secure with HttpOnly cookies
JWT	Signed token sent with each request (Authorization: Bearer ...)	Stateless; keep short-lived; do not store secrets inside
OAuth / SSO	Log in with Google, GitHub, etc.	Delegates identity to a provider

10.4 Common attacks and defences

Attack	What it is	Defence
SQL injection	Malicious input alters a SQL query	Parameterised queries, ORMs
XSS	Attacker injects script into your page	Escape output, avoid innerHTML, CSP header
CSRF	Tricks a logged-in user's browser into sending a request	CSRF tokens, SameSite cookies
Brute force	Guessing passwords repeatedly	Rate limiting, lockouts, MFA
Insecure secrets	Keys leaked in code or Git	Environment variables, secret managers

10.5 Security checklist

- Use HTTPS everywhere.
- Validate and sanitise ALL user input on the server.
- Set secure headers (helmet in Express).
- Use HttpOnly, Secure and SameSite flags on cookies.
- Keep dependencies updated (npm audit).
- Apply the principle of least privilege to database users and API keys.

- Do not reveal detailed error messages to users.

11. Frameworks, Tools and Workflow

11.1 Front-end frameworks and libraries

Tool	Purpose
React	Component-based UI library (huge ecosystem)
Vue	Approachable progressive framework
Angular	Full framework with TypeScript built in
Svelte	Compiler-based, minimal boilerplate
Next.js / Nuxt	Frameworks adding routing, server rendering and APIs
Tailwind CSS	Utility-first CSS framework
Bootstrap	Ready-made responsive components

11.2 React in a nutshell

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
export default Counter;
```

Key ideas: **components** (reusable pieces), **props** (inputs), **state** (data that changes), **hooks** (useState, useEffect), and a declarative UI that re-renders when state changes.

11.3 TypeScript

TypeScript adds static types to JavaScript, catching errors before runtime.

```
interface User { id: number; name: string; email?: string; }
function greet(user: User): string { return "Hi " + user.name; }
```

11.4 Tooling

- **VS Code** - editor; extensions: ESLint, Prettier, Live Server.
- **npm / pnpm / yarn** - package managers.
- **Vite** - fast dev server and bundler.
- **ESLint / Prettier** - find bugs and format code consistently.
- **Jest / Vitest / Playwright / Cypress** - unit and end-to-end testing.
- **Docker** - package apps with their environment for consistent deployment.
- **ORMs** (Prisma, Sequelize, Drizzle) - work with databases using code.

12. Deployment, Performance and SEO

12.1 Deployment options

Type	Examples
Static hosting	GitHub Pages, Netlify, Vercel, Cloudflare Pages
App platforms (PaaS)	Render, Railway, Fly.io, Heroku
Cloud / VPS	AWS, Google Cloud, Azure, DigitalOcean
Managed databases	PlanetScale, Supabase, MongoDB Atlas, AWS RDS

12.2 Deployment steps

- Push code to GitHub.
- Set environment variables on the host.
- Run the build (npm run build) and start command (npm start).
- Connect your domain; DNS records (A, CNAME) point it to the server.
- Enable HTTPS (most platforms do this automatically with Let's Encrypt).
- Set up monitoring and logging; automate with **CI/CD** (e.g. GitHub Actions).

12.3 Performance

- Compress and properly size images; use modern formats (WebP, AVIF) and lazy loading.
- Minify and bundle CSS and JS; remove unused code.
- Use caching (browser cache headers, CDN, Redis).
- Reduce the number and size of network requests.
- Use **defer** or **async** on scripts so they do not block rendering.
- Measure with Lighthouse and Core Web Vitals (LCP, INP, CLS).

```
<img src="photo.webp" alt="..." loading="lazy" width="600" height="400">
<script src="app.js" defer></script>
```

12.4 SEO basics

- Unique, descriptive **title** and **meta description** on each page.
- Semantic HTML and a clear heading structure.
- Fast, mobile-friendly, secure (HTTPS) pages.
- Meaningful URLs, a sitemap.xml and robots.txt.
- Open Graph tags for nice link previews when shared.

```
<meta name="description" content="Short summary of the page (about 150 chars)">
<meta property="og:title" content="Page title">
<meta property="og:image" content="https://example.com/cover.jpg">
```

13. Learning Roadmap and Cheat Sheets

13.1 Suggested roadmap

Stage	Learn	Build
1. Foundations	HTML, CSS, Git, how the web works	Personal portfolio page
2. Interactivity	JavaScript, DOM, fetch, JSON	To-do app, weather app
3. Layout and polish	Flexbox, Grid, responsive design, accessibility	Responsive landing page
4. Backend	Node and Express, REST, SQL / MySQL	Blog or notes API with database
5. Security and auth	Hashing, sessions or JWT, validation	Login and signup system
6. Frameworks	React (or Vue), TypeScript, Tailwind	Full-stack app
7. Ship it	Deployment, CI/CD, testing, performance	Deploy a real project online

13.2 Flexbox cheat sheet

Property	Common values
justify-content	flex-start, center, flex-end, space-between, space-around, space-evenly
align-items	stretch, flex-start, center, flex-end, baseline
flex-direction	row, row-reverse, column, column-reverse
flex-wrap	nowrap, wrap
gap	1rem, 10px 20px

13.3 Quick reference

Need to...	Use
Center a box horizontally	margin: 0 auto; (with a width)
Center anything	display: grid; place-items: center;
Stop overflow of images	img { max-width: 100%; height: auto; }
Select by id / class in JS	querySelector('#id') / querySelector('.class')
Prevent form reload	event.preventDefault()
Wait for a Promise	await inside an async function
Copy an object	{ ...obj }
Avoid SQL injection	Prepared statements with ? placeholders

13.4 Habits of good developers

- Build projects - tutorials alone are not enough.
- Read error messages carefully and learn to search effectively.

- Use version control from day one.
- Write readable code with clear names and small functions.
- Read documentation (MDN Web Docs is the gold standard).
- Keep learning; fundamentals outlast frameworks.

Useful resources: MDN Web Docs, web.dev, javascript.info, CSS-Tricks, freeCodeCamp, The Odin Project, W3Schools (for quick lookups), Can I use (browser support).